# **Reporte** Actividades **Práctico-Experimentales** Nro. 003

## 1. Datos Generales

| | |
|---|---|
| **Estudiantes:** | Wagner Balcazar<br>Ariana Cordova<br>Cristian Tomala<br>Jasiel Quezada |
| **Asignatura** | Sistemas Distribuidos |
| **Ciclo** | 6to |
| **Unidad** | Uno |
| **Resultado de aprendizaje de la unidad** | R2. Escribe un programa que realiza cualquier requerimiento de clasificación y conversión en unidades de mensajes, como paquetes, para comunicar información interesante entre dos hosts, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad. |
| **Práctica Nro.** | 3 |
| **Título de la Práctica** | Sincronización de Procesos: Exclusión Mutua, Semáforos y Problemas Clásicos de Concurrencia |
| **Nombre del Docente** | José O. Guamán Q. |
| **Fecha** | 06 de octubre de de 2026<br>09 de octubre de 2026 |
| **Horario** | Martes 10h30 a 11h30<br>Viernes 07h30 a 09h30 |
| **Lugar** | Aula |
| **Tiempo planificado en el Sílabo** | 3 horas |

## 2. Objetivo(s) de la Práctica:

Implementar una barrera de sincronización con variables de condición que detenga a los hilos hasta que todos alcancen el punto de encuentro.

- Implementar mecanismos de exclusión mutua utilizando mutex y cerrojos (locks) para proteger secciones críticas en programas multihilo, garantizando que no se produzcan condiciones de carrera al acceder a recursos compartidos.
- Aplicar semáforos como herramienta de sincronización para controlar el acceso a recursos limitados y coordinar la ejecución de hilos según restricciones específicas de orden y disponibilidad.
- Resolver el problema clásico del Productor-Consumidor utilizando buffers compartidos y mecanismos de sincronización que aseguren que

el productor no escriba en un buffer lleno y el consumidor no lea de un buffer vacío.

- Implementar la solución al problema de Lectores-Escritores, distinguiendo entre las variantes con prioridad para lectores y prioridad para escritores, analizando las implicaciones de cada enfoque en términos de inanición y concurrencia.

## 3. Materiales y reactivos:

Guia practica Lista

de ejercicios

## 4. Equipos y herramientas

Hardware

- Computador

Software

- Lenguaje de programación (Python, java sprint boot, node)
- IDE de programación
- Archivo de texto

## 5. Procedimiento / Metodología

- **Análisis**

En esta fase cada uno de los integrantes del grupo analizó su ejercicio correspondiente y así mismo, se procedió a instalar las tecnologías necesarias para el desarrollo de esta práctica.

**Procedimiento Ejercicio 1**

1. **Identifique el recurso compartido.**

   El recurso compartido vendrían siendo los espacios para estacionar los vehículos.
2. **Determine la cantidad máxima de vehículos que pueden permanecer dentro del estacionamiento.**

   La capacidad máxima que el estacionamiento puede ofrecer es de 4 espacios.
3. **Cree un semáforo contador inicializado en 4.**

   Los primeros cuatro vehículos que consigan un permiso podrán ingresar. Los restantes quedarán bloqueados al intentar adquirir un permiso mientras no haya espacios disponibles. Cuando un vehículo salga, otro podrá continuar.
4. **Cree un proceso vehiculo(id).**
5. **El vehículo deberá solicitar un espacio antes de ingresar.**

6. **Si no existen espacios disponibles, deberá permanecer bloqueado.**
7. **Una vez que obtenga un espacio, deberá simular su permanencia.**
8. **Al salir, deberá liberar el espacio mediante signal().**
9. **Cree 10 procesos vehículo.**
10. **Analice qué sucede cuando llegan más de 4 vehículos simultáneamente.**

Los primeros cuatro vehículos que consigan un permiso podrán ingresar. Los restantes quedarán bloqueados al intentar adquirir un permiso mientras no haya espacios disponibles. Cuando un vehículo salga, otro podrá continuar.

**Pseudocódigo**

```
SEMÁFORO espacios = 4


PROCEDIMIENTO vehiculo(id)
   escribir("Vehículo ", id, " llega al estacionamiento")

   wait(espacios)

   escribir("Vehículo ", id, " ingresa al estacionamiento") estacionar(id)

   esperar_tiempo()

   escribir("Vehículo ", id, " sale del estacionamiento")

   signal(espacios)

FIN PROCEDIMIENTO


INICIO

   PARA i = 1 HASTA 10 HACER
      crear_proceso(vehiculo, i) FIN
   PARA

FIN
```

**Pregunta de análisis:**

**¿Qué ocurriría si el semáforo se inicializara en 1? ¿Y si se inicializara en 10?**

Si el semáforo se inicializa en 1 los vehiculo se puede estacionar en uno de los lugares disponibles, en caso que se inicialice en 10, 4 de los autos se llegarian a estacionar y los otros estarian en espera.

**Ejercicio 2. Cajero automático compartido — Exclusión mutua**

**Objetivo**: Implementar exclusión mutua para evitar condiciones de carrera sobre un recurso compartido.

**Situación**:
Una cuenta bancaria tiene un saldo inicial de $1000. Tres procesos realizan operaciones simultáneamente sobre la misma cuenta:

**Condición:**
Usarse varias veces sin que el valor varie

- Proceso 1: retiro de $300.
- Proceso 2: retiro de $250.
- Proceso 3: depósito de $200.

El saldo debe modificarse de forma segura.

Procedimiento

1. Identifique la variable compartida.
   La variable compartida es saldo debido a que se maneja en la clase de cuentabancariservice, recurso crítico para los tres procesos a generar.
2. Determine qué operaciones pueden ejecutarse simultáneamente.
   Las operaciones a ejecutar de manera simultánea son la operación de retirar y depositar.
3. Cree un mutex para proteger el saldo.
   El mutex implementado para el codigo sera el sig:
   private final ReentrantLock mutexSaldo = new ReentrantLock ();
4. Antes de consultar o modificar el saldo, el proceso debe adquirir el mutex.
   Se logra al ejecutar el comando mutexSald.lock(); al inicio de los métodos de retirar y depositar.
5. Realice la operación correspondiente.
6. Libere el mutex después de actualizar el saldo.
7. Cree los tres procesos.
8. Ejecute el programa varias veces.
9. Analice qué podría ocurrir si se elimina el mutex.
10. Compare el comportamiento con y sin exclusión mutua.

**Pseudocódigo**

```
saldo = 1000
MUTEX mutex_saldo = 1
PROCEDIMIENTO retirar(id, cantidad) lock(mutex_saldo)
   escribir("Proceso ", id, " intenta retirar $", cantidad)

   SI saldo >= cantidad ENTONCES
      saldo = saldo - cantidad
      escribir("Retiro realizado") escribir("Saldo
      actual: $", saldo)
   SINO
```

```
        escribir("Saldo insuficiente")
    FIN SI
    unlock(mutex_saldo) FIN
PROCEDIMIENTO

PROCEDIMIENTO depositar(id, cantidad)
    lock(mutex_saldo)
    escribir("Proceso ", id, " deposita $", cantidad)
    saldo = saldo + cantidad
    escribir("Saldo actual: $", saldo) unlock(mutex_saldo)
FIN PROCEDIMIENTO
INICIO
    crear_proceso(retirar, 1, 300)
    crear_proceso(retirar, 2, 250)
    crear_proceso(depositar, 3, 200)
FIN
```

**Pregunta de análisis:**

¿Por qué el acceso a saldo debe protegerse con un mutex?

Debido a que se trata de una instancia crítica, siendo que si no se protege, puede generar una pérdida de dinero o un aumento sin sentido del mismo.

## Ejercicio 3. Fábrica de ensamblaje — Productor-Consumidor

**Objetivo:** Implementar el patrón productor-consumidor utilizando una cola limitada y semáforos.

**Situación:**

Una fábrica tiene una línea de producción con capacidad para almacenar **5 piezas** temporalmente.

- Los productores fabrican piezas.
- Los consumidores toman piezas para ensamblarlas.
- Un productor no puede colocar una pieza si el almacén está lleno.
- Un consumidor no puede retirar una pieza si el almacén está vacío.

**Procedimiento**

1. Cree una cola con capacidad máxima de 5 elementos.
2. Cree un semáforo espacios inicializado en 5.
3. Cree un semáforo elementos inicializado en 0.
4. Cree un mutex para proteger la cola.
5. Implemente el proceso productor.
6. El productor deberá esperar un espacio disponible.
7. Debe adquirir el mutex antes de modificar la cola.
8. Debe insertar una pieza.
9. Debe liberar el mutex.
10. Debe indicar que existe una nueva pieza.
11. Implemente el proceso consumidor.
12. El consumidor deberá esperar hasta que exista una pieza.
13. Debe adquirir el mutex antes de retirar una pieza.
14. Debe retirar la pieza.

15. Debe liberar el mutex.
16. Debe indicar que existe un espacio disponible.

**Pseudocódigo**

```
COLA     almacen
CAPACIDAD = 5

SEMÁFORO espacios = 5
SEMÁFORO elementos = 0

MUTEX mutex_almacen = 1


PROCEDIMIENTO productor(id)

    REPETIR 5 VECES

     pieza = fabricar_pieza()

     wait(espacios)

     lock(mutex_almacen)

     insertar(almacen, pieza)

     escribir("Productor ", id,
           " agregó pieza ", pieza)

     unlock(mutex_almacen)

     signal(elementos)

   FIN REPETIR

  FIN PROCEDIMIENTO


PROCEDIMIENTO consumidor(id)

    REPETIR 5 VECES

     wait(elementos)
```

```
        lock(mutex_almacen) pieza

        = extraer(almacen)

        escribir("Consumidor ", id, "
              retiró pieza ", pieza)

        unlock(mutex_almacen)

        signal(espacios)

        ensamblar(pieza)

    FIN REPETIR

FIN PROCEDIMIENTO


INICIO

    PARA i = 1 HASTA 2 HACER
        crear_proceso(productor, i)
    FIN PARA

    PARA i = 1 HASTA 2 HACER
        crear_proceso(consumidor, i)
    FIN PARA

FIN
```

**Pregunta de análisis:**

**¿Qué sucede si espacios se inicializa en 0? ¿Qué sucede si elementos se inicializa en 5?**

SI espacios se inicializa en 0 y elementos en 5, la lógica aplicada intentaría leer los datos que no existen (arrancar con elementos=5), ya que el semáforo le daria paso inmediato al hilo consumidor para que ejecute la instrucción de remover objeto pero como la lista interna en la memoria RAM esta fisicamente vacía, se intentara acceder a la posición 0 de un arreglo de tamaño 0 y lanza un a excepción, por lo que se producirá un error de contradicción entre el conteo del semáforo y el contenido real de la memoria

**Ejercicio 4. Cruce de vehículos — Control de acceso en una dirección**

**Objetivo:** Coordinar procesos que comparten un recurso y evitar que dos grupos incompatibles accedan simultáneamente.

**Situación:**

Un puente permite circular en ambos sentidos, pero solamente puede existir **una dirección de circulación activa a la vez**.

Pueden existir varios vehículos cruzando hacia el norte, pero no puede haber vehículos hacia el norte y hacia el sur simultáneamente.

**Procedimiento**

1. Identifique el recurso compartido.
2. Defina una variable que indique la dirección actual.
3. Mantenga un contador de vehículos que están cruzando.
4. Proteja las variables mediante un mutex.
5. Un vehículo deberá verificar la dirección antes de entrar.
6. Si existen vehículos en dirección contraria, deberá esperar.
7. Si puede entrar, incrementará el contador correspondiente.
8. Después de cruzar, disminuirá el contador.
9. Cuando no queden vehículos, el puente podrá cambiar de dirección.
10. Analice el problema de inanición.

**Pseudocódigo**

```
MUTEX mutex = 1
direccion = NINGUNA
vehiculos_norte = 0
vehiculos_sur = 0
PROCEDIMIENTO vehiculo_norte(id) lock(mutex)

  MIENTRAS direccion == SUR HACER esperar()
  FIN MIENTRAS
  direccion = NORTE
  vehiculos_norte = vehiculos_norte + 1 unlock(mutex)

  escribir("Vehículo ", id,
        " cruza hacia el NORTE")

  cruzar_puente(id)

  lock(mutex)
  vehiculos_norte = vehiculos_norte - 1
  SI vehiculos_norte == 0 ENTONCES
    direccion = NINGUNA
  FIN SI
```

unlock(mutex)
FIN PROCEDIMIENTO

El estudiante deberá completar el procedimiento: PROCEDIMIENTO
vehiculo_sur(id)
siguiendo la misma lógica.
**Reto:** modificar el algoritmo para evitar que una dirección monopolice el puente.

**Ejercicio 5. Centro de impresión — Sincronización de procesos**

**Objetivo:** Utilizar semáforos para establecer un orden determinado de ejecución entre procesos.

**Situación:**
Un centro de impresión tiene tres procesos:

- Diseño
- Revisión
- Impresión

Para cada documento debe cumplirse obligatoriamente: DISEÑO → REVISIÓN → IMPRESIÓN

No se puede revisar un documento que todavía no ha sido diseñado y tampoco se puede imprimir antes de que haya sido revisado.

**Procedimiento**

1. Identifique los procesos participantes.
2. Cree un semáforo para cada etapa.
3. Permita inicialmente que el proceso de diseño pueda ejecutarse.
4. Diseño deberá habilitar a revisión.
5. Revisión deberá habilitar a impresión.
6. Impresión deberá habilitar nuevamente a diseño.
7. Repita el ciclo para 3 documentos.
8. Verifique que el orden siempre sea correcto.
9. Analice qué ocurre si se elimina uno de los signal().

**Pseudocódigo**

SEMÁFORO turno_diseno = 1
SEMÁFORO turno_revision = 0
SEMÁFORO turno_impresion = 0

PROCEDIMIENTO diseno()

```
        REPETIR 3 VECES
          wait(turno_diseno)
          escribir("Diseño prepara documento")
          crear_documento() signal(turno_revision)
        FIN REPETIR
      FIN PROCEDIMIENTO

PROCEDIMIENTO revision()
        REPETIR 3 VECES
          wait(turno_revision)
          escribir("Revisión verifica documento")
          revisar_documento() signal(turno_impresion)
        FIN REPETIR
FIN PROCEDIMIENTO

PROCEDIMIENTO impresion()
        REPETIR 3 VECES
          wait(turno_impresion)
          escribir("Impresión imprime documento")
          imprimir_documento() signal(turno_diseno)
```

```
    FIN REPETIR

FIN PROCEDIMIENTO


INICIO

    crear_proceso(diseno)

    crear_proceso(revision)

    crear_proceso(impresion)

FIN
```

## 6. Resultados esperados:

**REPOSITORI0:**[https://github.com/WagnerBalcazar/APE3-SISTEMAS-DISTRIBUIDOS/tree/develop/sistemas-distribuidos-practica](https://github.com/WagnerBalcazar/APE3-SISTEMAS-DISTRIBUIDOS/tree/develop/sistemas-distribuidos-practica)

**VIDEO:** [https://youtu.be/vsC7aYxq5m8](https://youtu.be/vsC7aYxq5m8)

Dentro del desarrollo de esta práctica se desarrollaron los 5 microservicios cada uno correspondiente a cada ejercicio, las tecnología que se utilizo fue spring boot para el backend y react para el front.




*Fig.1 Microservicios.*

- **Ejercicio 1**

El ejercicio consiste en simular el funcionamiento del control de acceso de un estacionamiento que cuenta con una capacidad máxima de cuatro vehículos. Durante la simulación, diez vehículos llegan de manera concurrente e intentan ingresar al establecimiento, por lo que es necesario organizar su entrada para evitar que se supere el número de espacios disponibles.

Cuando un vehículo llega, se verifica si existe un espacio libre para permitirle el ingreso. Si el estacionamiento tiene capacidad disponible, el vehículo puede entrar y ocupar un lugar; en cambio, si todos los espacios están ocupados, debe esperar hasta que otro vehículo salga y deje un espacio libre. Una vez que un vehículo termina su estancia, abandona el estacionamiento y permite que otro vehículo pueda ingresar.

El propósito del ejercicio es representar una situación real en la que varios vehículos intentan acceder a un recurso limitado al mismo tiempo. De esta manera, se busca garantizar que el estacionamiento mantenga un máximo de cuatro vehículos en su interior, que los espacios disponibles se administren correctamente y que los vehículos que esperan puedan ingresar conforme se liberen lugares.

El microservicio 1 sigue un patrón de diseño modelo vista controlador, lo cual organiza el código con sus funcionalidades correspondientes.




*Fig.2 Estado inicial ejercicio 1*




*Fig.3 Simulación de 10 autos para estacionamiento.*

En la figura 2 se puede observar que 4 autos ocupan los lugares de estacionamiento y los 6 restantes estan en una cola de espera, para ubicarse en los lugares restantes.

- **Ejercicio 2**

El ejercicio 2 consiste en la simulación de un sistema de cajero automático compartido para la gestión segura de un recurso básico con respecto al acceso concurrente de múltiples procesos (retiros y depósitos).
Para llevar esto a cabo, se centralizó el estado y el control de acceso al servicio de la cuenta bancaria (CuentaBancariaServices.java) es por medio de singleton, asegurando la existencia de una única instancia compartida en memoria (un saldo de $1000) de la cual todos los procesos están obligados a interactuar con la misma instancia.
Finalmente para mantener un acceso seguro y concurrente se implementó un bloque mutex (ReentrantLock) esto con el fin de garantizar una exclusión mutua estricta, con la finalidad que sólo un hilo a la vez pueda entrar a la sección crítica para modificarla, bloqueando las demás hasta que la operación finalice.

Por medio de la simulación, se lanzan tres hilos de manera simultánea (un retiro de $500, otro retiro de $250 y un depósito de $200), antes de generar cualquier cambio, los hilos deben intentar acceder al candado del mutex, caso de que la cuenta sea modificada por un hilo, los demas estaran retenidos en un tiempo de espera, al momento que el hilo activo termine, liberará el acceso, permitiendo que el siguiente hilo en espera genere su ejecución, garantizando una prevención en la sobrescritura de datos, siendo el saldo final exacto de $650.
Como se mencionó anteriormente, el patrón de dise;o usado se trata de Singleton, para la gestión automática del contenedor de inversión de control del framework de Spring Boot, por medio del @Service generando una única instancia en la memoria y por medio de las dependencia, generará el proceso de las peticiones de los hilos.




*Fig.4 Simulación del cajero junto a los hilos propuestos.*

En la figura 3, se evidencia el manejo de los hilos como también el contexto del uso del Mutex para generar un proceso sólido y lineal para el procesamiento de los procesos que generara el cajero automático.

- **Ejercicio 3**

El ejercicio 3 consiste en simular el patrón de sincronización Productor-Consumidor para una mejor gestión de capacidad de un almacén compartido de forma concurrente y segura entre múltiples hilos
Por lo que se centralizó el estado y control de acceso del almacén en la clase Almacen.java, encargada de gestionar los productos almacenados, la capacidad disponible y el acceso exclusivo a la estructura de datos.
Se aplica una estructura de datos compartida (como una lista o arreglo) para guardar los productos y una variable que registra la cantidad actual almacenada. Para coordinar el acceso concurrente entre productores y consumidores se implementa un mecanismo basado en semáforos, contadores y exclusión mutua.

- Semaphore emptySlots: controla los espacios libres disponibles para producir.
- Semaphore fullSlots: controla los productos disponibles para consumir

- ReentrantLock (Mutex): garantiza la exclusión mutua para modificar la estructura de datos sin interferencia de otros hilos

Antes de producir un hilo se verifica si existen espacios disponibles llamando a emtySlots.acquire()-, si la bodega esta llena el hilo quedará retenido automáticamente. De forma análoga antes de consumir un hilo verifica si hay productos disponibles con fullSlots.acquire(); si la bodega está vacía el hilo debe esperar.

Además se considera que para evitar el bloqueo mutuo, los permisos de los semáforos se adquieren fuera del bloqueo de exclusión mutua. Una vez obtenido el permiso del semáforo el hilo adquiere el candado, inserta o remueve el producto de forma segura y libera el candado inmediatamente. Al finalizar la operación se notifica el cambio liberando el semáforo opuesto tras el producto tras consumir, permitiendo que los hilos en espera reanuden su ejecución.

El microservicio 3 creado para este ejercicio sigue la arquitectura en capas del patrón Modelo-Vista-Controlador (MVI) ayudando asi a organizar la lógica de concurrencia en la cada servicio, la explosicion de la API REST en el contador y la interfaz gráfica defensiva en el frontend

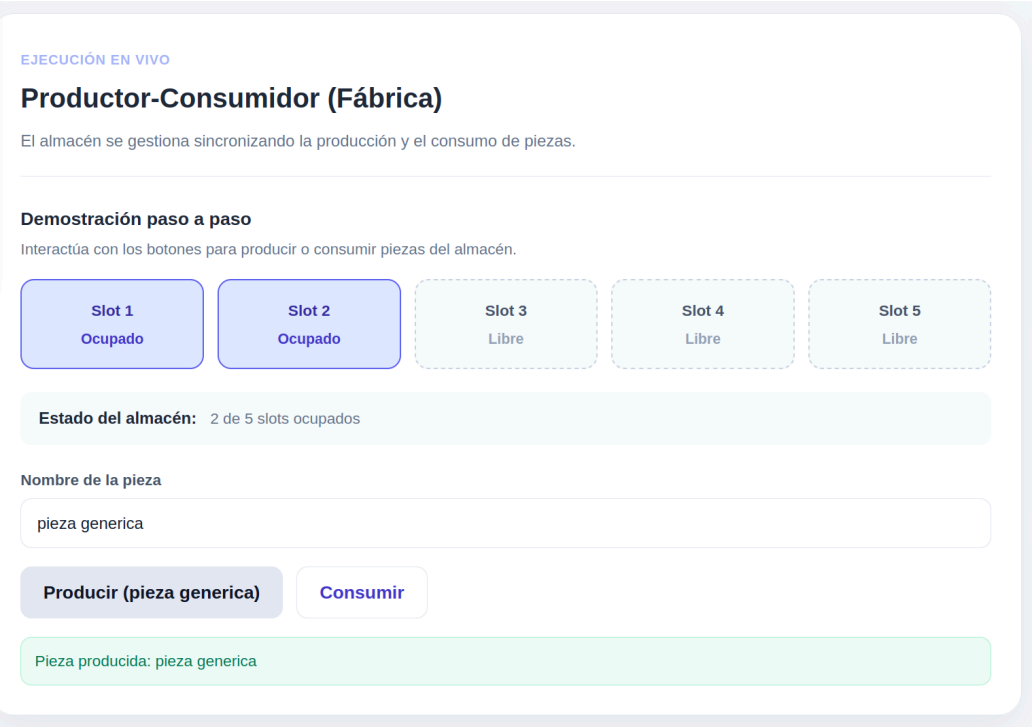



*Fig.5 Generación de dos piezas que ocupan 2 de 5 espacios del almacén*

En la figura 4 se observa la producción de 2 piezas dentro del almacén, con las opciones de seguir produciendo piezas o listas para ser consumidas por el consumidor.

- **Ejercicio 4**

El ejercicio consiste en simular un puente por el que pueden circular vehículos en ambos sentidos. Sin embargo, todos los vehículos que se encuentren cruzando deben hacerlo en la misma dirección, ya sea NORTE o SUR. Por tanto, es necesario controlar el acceso al puente para evitar conflictos entre procesos concurrentes.

La solución se basa en centralizar el estado del puente en la clase PuenteServicio.java, encargada de gestionar la dirección activa, los vehículos que están cruzando y las solicitudes en espera.

Se utiliza el enumerado Direccion.java, cuyos valores son NINGUNA, NORTE y SUR. Además, se mantienen dos contadores, vehiculosNorte y vehiculosSur, para registrar los vehículos que están cruzando en cada sentido.

Para proteger las variables compartidas se implementa un mecanismo de

sincronización mediante ReentrantLock y Condition. Antes de ingresar, cada vehículo verifica si la dirección del puente es compatible con su sentido de circulación. Si existe tráfico en la dirección contraria, debe esperar hasta que el puente quede libre.
Cuando un vehículo termina de cruzar, se reduce el contador correspondiente. Si ya no quedan vehículos, la dirección vuelve a NINGUNA y se notifica a los procesos que están esperando, permitiendo que se realicen nuevos ingresos.
El cruce se ejecuta fuera del bloqueo para permitir que varios vehículos de una misma dirección avancen simultáneamente, sin necesidad de procesarlos uno por uno.
El microservicio(microservicio 4) creado para este ejercicio sigue el patrón Modelo–Vista–Controlador (MVC), que organiza el código según sus responsabilidades.




*Fig.6 Auto en dirección Norte cruzando*

En la Figura 5 se observa que un vehículo está cruzando el puente en dirección norte, mientras que otro vehículo intenta ingresar desde el sur. Debido a que el puente solo permite la circulación en una dirección a la vez, el segundo vehículo debe esperar hasta que el primero termine de cruzar y el puente quede libre para cambiar de sentido.

El ejercicio permitió aplicar conceptos de sincronización y concurrencia en un sistema distribuido mediante la gestión de un recurso compartido. El uso de bloqueos, condiciones y contadores garantiza que los vehículos de sentidos opuestos no utilicen el puente simultáneamente.

- **Ejercicio 5**

El centro de impresión está compuesto por tres procesos: Diseño, Revisión e Impresión. Cada documento debe seguir obligatoriamente la secuencia DISEÑO -> REVISIÓN -> IMPRESIÓN. Por lo tanto, no se puede revisar un documento antes de diseñarlo ni imprimirlo sin haber completado la revisión.
Para garantizar este orden, se utilizan semáforos que regulan cuándo puede ejecutarse cada etapa.
Se implementaron tres semáforos en la clase CentroImpresionServicio.java, uno para cada proceso:

- turnoDiseno: inicia con valor 1, permitiendo que Diseño comience.
- turnoRevision: inicia con valor 0, bloqueando la revisión hasta que Diseño

termine.
- turnoImpresion: inicia con valor 0, impidiendo la impresión hasta que Revisión complete su trabajo.

El funcionamiento se basa en las operaciones acquire() y release(), equivalentes a las operaciones de espera (wait) y señalización (signal).
El proceso de Diseño prepara el documento y habilita a Revisión. A continuación, Revisión verifica el contenido y permite que Impresión continúe. Finalmente, Impresión completa el trabajo y vuelve a habilitar a Diseño para comenzar con el siguiente documento.
Este ciclo se repite para tres documentos, aunque la cantidad puede modificarse mediante el endpoint correspondiente. De esta manera, los semáforos garantizan el orden de ejecución sin depender de la coordinación manual del cliente.




*Fig.7 Documento en Diseño*

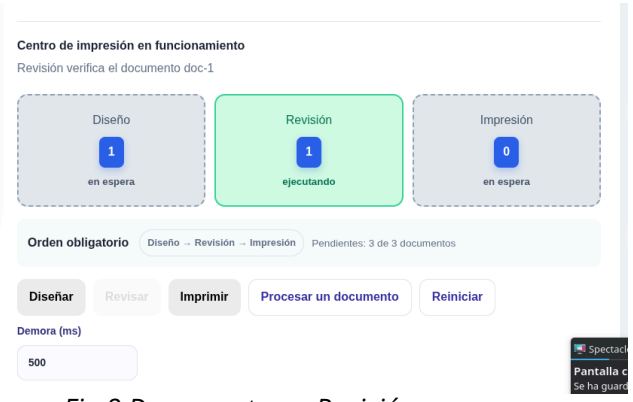



*Fig.8 Documento en Revisión*

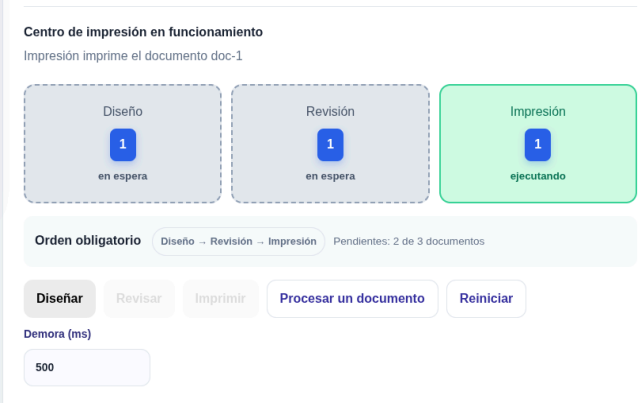



*Fig.9 Documento en Impresión*

En las Figuras 6, 7 y 8 se muestran las diferentes etapas por las que pasa un documento dentro del centro de impresión. En la Figura 6, el documento se encuentra en la fase de Diseño, donde se prepara y se genera el contenido inicial. Una vez finalizada esta etapa, el sistema habilita automáticamente el proceso de Revisión.
En la Figura 7, se observa el documento en la etapa de Revisión, en la cual se verifica que el contenido haya sido preparado correctamente antes de continuar. Esta fase solo puede ejecutarse cuando Diseño ha finalizado, debido a la sincronización establecida mediante semáforos.
Finalmente, en la Figura 8, el documento se encuentra en la etapa de Impresión, donde se completa el proceso. Para llegar a esta fase, es necesario que las etapas anteriores hayan terminado en el orden establecido.
Las figuras permiten comprobar visualmente el funcionamiento del mecanismo de sincronización, ya que cada documento avanza de forma ordenada por las tres etapas: DISEÑO -> REVISIÓN > IMPRESIÓN. De esta manera, los semáforos impiden que una fase comience antes de que la anterior haya concluido, garantizando la

secuencia correcta del proceso y evitando errores de coordinación entre los distintos hilos de ejecución.

El ejercicio permitió aplicar semáforos como mecanismo de sincronización para coordinar procesos concurrentes. La solución garantiza que cada documento pase por las etapas de diseño, revisión e impresión en el orden establecido.
Además, las pruebas automatizadas confirmaron el funcionamiento del ciclo para tres documentos y permitieron comprobar que la ausencia de una señal interrumpe la ejecución.

## 7. Preguntas de Control:

- **¿Qué es una condición de carrera?**

  Se trata de un estado en el cual múltiples hilos o procesos acceden e intentan modificar un recurso compartido al mismo tiempo, generando un resultado que depende del orden que el sistema ejecuta las instrucciones generando datos inconsistentes a la instancia o recurso modificado.

- **Cuál es la diferencia fundamental entre un semáforo binario y un mutex? ¿Pueden usarse indistintamente? Justifique su respuesta con al menos dos diferencias prácticas**
  La diferencia entre los dos radica en el concepto, el mutex genera un bloqueo de candado generando autorización y luego obliga a liberar el candado, mientras que un semáforo binario se trata de un mecanismo simple de señalización por medio de un contador.
  Estos no pueden ser usados sin darle importancia, ya que manejan diferentes aspectos, mutex restringe con la finalidad de proteger secciones críticas siendo que un solo hilo modifique una sola variable a la vez, mientras que el semáforo es usado para la notificación o sincronización de eventos.

- **Explique qué es un interbloqueo (deadlock) y enuncie las cuatro condiciones necesarias de Coffman. ¿Cuál de estas condiciones se elimina al usar la estrategia de ordenamiento de recursos?**
  EL interbloqueo es un estado de bloqueo permanente, donde un conjunto de procesos quedan esperando de manera indefinida, esto debido al manejo de procesos: cada proceso retiene un recurso y solicita al mismo tiempo otro recurso que es retenido por otro proceso.
  las cuatro condiciones de Coffman son:
  - **Exclusión mutua:** un recurso debe ser retenido sin ser compartido.
  - **Retención y espera:** Un proceso que retiene recursos puede solicitar nuevos recursos y esperar por ellos.
  - **Sin expropiación (No preemption)**: Los recursos no deben ser quitados a la fuerza, solo el proceso que los retiene puede liberarlos voluntariamente.
  - **Espera circular**: Se trata de una cadena cerrada de procesos donde cada uno demanda recursos almacenados del siguiente.

## 8. Conclusiones

- Se analizó la importancia de las variables de condición, los mecanismos de bloqueo y los contadores para coordinar de manera segura el acceso a los recursos compartidos, evitando condiciones de carrera y garantizando una ejecución ordenada de los procesos concurrentes.
- Además, el modelado de problemas clásicos y su impacto en sistemas reales nos ayudó a comprender mejor la gestión de recursos compartidos, ya que los ejercicios consistió en aplicar semáforos y mecanismos de bloqueo para evitar que los procesos quedaran bloqueados indefinidamente y mantener la consistencia del sistema.
- Se integró la arquitectura de microservicios con Docker Compose y un API Gateway demostrando la efectividad de desacoplar los modulos del sistema permitiendo que cada servicio gestione su propia lógica de concurrencia de forma independiente sin comprometer la estabilidad del sistema.

## 9. Recomendaciones

- Investigar a fondo la arquitectura de microservicios que se va a implementar, ya que, en ocasiones, creemos tener un conocimiento suficiente sobre su funcionamiento, pero al momento de aplicarla surgen dudas o dificultades que requieren una investigación más profunda. Por ello, es importante comprender sus principios, componentes, ventajas y desafíos antes de comenzar el desarrollo, para tomar decisiones técnicas adecuadas y evitar errores durante la implementación.
- Se recomienda realizar pruebas y análisis de rendimiento para las soluciones implementadas.
- Se recomiendo mantener una estrategia y comunicación clara entre los integrantes de grupo para poder avanzar sin complicaciones y bajo una planificación grupal en donde cada integrante aporta de manera efectiva

## 10. Bibliografía

Semana 3. Guaman, Jose. 2026

## 11. Declaración de IA

Declaró que, para la elaboración de esta tarea, hice uso de una herramienta de Inteligencia Artificial (Gemini, Notebook) únicamente como apoyo para mejorar la redacción y síntesis de ideas.

El uso de la IA se realizó respetando el Contrato de uso ético y responsable de la Inteligencia Artificial, comprometiéndose a:

No copiar directamente sin comprensión de los textos generados.

Revisar, adaptar y redactar las ideas en nuestras propias palabras.

Mantener un aprendizaje activo y crítico en el desarrollo del trabajo

**Promt Utilizado:**

Actúa como tutor de programación. Te compartiré mi avance de la práctica. Revisa mi código y mi informe, señala mis errores (lógica, concurrencia, condiciones de carrera, interbloqueos, redacción) y guíame paso a paso para corregirlos.